

Algorithm Performance & Transferability: 8-Model Tournament

Train models, evaluate PR AUC, score ACS, execute sanity checks, and
auto-export

PURPOSE

```
# =====  
# PURPOSE  
# =====  
# Model Comparison Workflow:  
#   1. Define global parameters and feature sets.  
#   2. Train 8 diverse ML models on the SIPP donor sample.  
#   3. Evaluate diagnostics across In-Sample, Out-of-Sample, and Drift contexts.  
#   4. Validate thresholds on the target-like test set.  
#   5. Score the ACS Target Sample with the winning model.  
#   6. Generate Sanity Checks (Percentiles, Lift, Feature Importance).  
#   7. Automatically export ONLY the highest-performing model and lock environment.  
# =====  
  
report_timestamp <- format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z")  
cat("Report rendered at:", report_timestamp, "\n")
```

Report rendered at: 2026-07-28 14:45:45 PDT

1. SETUP & PATHS

```
# =====  
# 1. SETUP & PATHS
```

```
# =====
library(here)
library(dplyr)
library(tidyr)
library(readr)
library(caret)
library(ROCR)
library(MLmetrics)
library(knitr)
library(pROC)

# Model Libraries
library(catboost)
library(xgboost)
library(lightgbm)
library(ranger)
library(ada)

# Paths
if (Sys.info()["sysname"] == "Windows") {
  base_dir <- "G:/Shared drives/Undocu Research"
} else {
  base_dir <- "~/Library/CloudStorage/GoogleDrive-vsovero@ucr.edu/Shared drives/Undocu Research"
}

data_path <- file.path(base_dir, "Data")
ml_output_path <- here("ML Outputs")
dir.create(ml_output_path, showWarnings = FALSE, recursive = TRUE)

acs_prediction_export <- file.path(ml_output_path, "ACS_SIPP_gbm.csv")
```

2. READ AND CLEAN BASE DATA

```
# =====
# 2. READ AND CLEAN BASE DATA
# =====
# DATA FILTERS APPLIED:
# 1. Base Universe: SIPP 2008 Wave 2 and ACS Target Sample
# 2. Age Constraint: 18 to 55 years old (Prime working age)
# 3. Employment: Currently employed (employed == 1)
```

```

# 4. Missingness: Dropped rows missing core demographic features
# 5. Target Variable: undocu_likely standardized to numeric 1 (Yes) or 0 (No)
# =====

sipp_file <- file.path(data_path, "(Step 1 output) Core_TM SIPP 2008 Wave 2.csv")
acs_file <- file.path(data_path, "ACS Target Sample.csv")

sipp_raw <- read_csv(sipp_file, show_col_types = FALSE)
acs_raw <- read_csv(acs_file, show_col_types = FALSE)

features <- c(
  "age", "fem", "married", "cit_spouse", "medicaid", "nonfluent",
  "spanish_hispanic_latino", "central_latino", "bpl_asia",
  "household_size", "poverty", "asian", "black", "white",
  "other_race", "years_us", "years_us_missing", "yrsted"
)

sipp <- sipp_raw %>%
  mutate(undocu_likely = ifelse(undocu_likely %in% c("X1", "1", 1), 1, 0)) %>%
  filter(age >= 18, age <= 55) %>%
  drop_na(all_of(features), undocu_likely)

acs <- acs_raw %>% drop_na(all_of(features))

# Split SIPP
set.seed(1)
train_index <- createDataPartition(sipp$undocu_likely, p = 0.70, list = FALSE)
sipp_train <- sipp[train_index, ]
sipp_test <- sipp[-train_index, ]

# Define Target-Like Validation Set
sipp_targetlike <- sipp_test %>%
  filter(age >= 22, age <= 55)

```

3. FORMAT DATA STRUCTURES FOR DIVERSE MODELS

```

# =====
# 3. FORMAT DATA STRUCTURES FOR DIVERSE MODELS
# =====
# Extract features and targets

```

```

x_train <- sipp_train %>% select(all_of(features))
y_train_num <- as.numeric(sipp_train$undocu_likely)

x_valid <- sipp_targetlike %>% select(all_of(features))
y_valid_num <- as.numeric(sipp_targetlike$undocu_likely)

x_acs <- acs %>% select(all_of(features))

# A. CatBoost Pools
pool_train <- catboost.load_pool(data = x_train, label = y_train_num)
pool_valid <- catboost.load_pool(data = x_valid, label = y_valid_num)
pool_acs <- catboost.load_pool(data = x_acs)

# B. Caret Dataframes (Require Factor Targets)
caret_train <- x_train %>%
  mutate(undocu_likely = factor(
    y_train_num,
    levels = c(1, 0), labels = c("Class1", "Class0")
  ))
caret_valid <- x_valid %>%
  mutate(undocu_likely = factor(
    y_valid_num,
    levels = c(1, 0), labels = c("Class1", "Class0")
  ))
caret_acs <- x_acs

# C. One-Hot Matrices (XGBoost & LightGBM)
dummy_model <- dummyVars(~., data = x_train)
mat_train <- predict(dummy_model, newdata = x_train)
mat_valid <- predict(dummy_model, newdata = x_valid)
mat_acs <- predict(dummy_model, newdata = x_acs)

# XGBoost DMatrices
dtrain_xgb <- xgb.DMatrix(data = mat_train, label = y_train_num)
dvalid_xgb <- xgb.DMatrix(data = mat_valid, label = y_valid_num)
dacs_xgb <- xgb.DMatrix(data = mat_acs)

# LightGBM Datasets
lgb_train <- lgb.Dataset(data = mat_train, label = y_train_num)

```

4. TRAIN THE 8 MODELS

```
# =====
# 4. TRAIN THE 8 MODELS (GRID SEARCH OPTIMIZED)
# =====

models <- list()
caret_ctrl <- trainControl(
  method = "cv", number = 5,
  summaryFunction = prSummary, classProbs = TRUE
)

# 1. CatBoost (Grid Search)
set.seed(1)
catboost_grid <- expand.grid(
  learning_rate = c(0.01, 0.05), depth = c(4, 6), l2_leaf_reg = c(3, 5)
)
best_cb_pr <- 0
best_cb_params <- list()
for (i in 1:nrow(catboost_grid)) {
  cv_res <- catboost.cv(
    pool = pool_train,
    params = list(
      loss_function = "Logloss", eval_metric = "PRAUC",
      learning_rate = catboost_grid$learning_rate[i],
      depth = catboost_grid$depth[i],
      l2_leaf_reg = catboost_grid$l2_leaf_reg[i],
      iterations = 500, od_type = "Iter", od_wait = 50,
      logging_level = "Silent"
    ),
    fold_count = 5
  )
  if (max(cv_res$test.PRAUC.mean) > best_cb_pr) {
    best_cb_pr <- max(cv_res$test.PRAUC.mean)
    best_cb_params <- catboost_grid[i, ]
  }
}
models$CatBoost <- catboost.train(
  learn_pool = pool_train,
  params = list(
    loss_function = "Logloss", eval_metric = "PRAUC",
    learning_rate = best_cb_params$learning_rate,
    depth = best_cb_params$depth,
```

```

    l2_leaf_reg = best_cb_params$l2_leaf_reg,
    iterations = 500, logging_level = "Silent"
  )
)

# 2. XGBoost (Grid Search)
set.seed(1)
xgb_grid <- expand.grid(eta = c(0.01, 0.05), max_depth = c(4, 6))
best_xgb_pr <- 0
best_xgb_params <- list()
for (i in 1:nrow(xgb_grid)) {
  cv_res <- xgb.cv(
    params = list(
      objective = "binary:logistic", eval_metric = "aucpr",
      eta = xgb_grid$eta[i], max_depth = xgb_grid$max_depth[i]
    ),
    data = dtrain_xgb, nrounds = 500, nfold = 5,
    early_stopping_rounds = 50, maximize = TRUE, verbose = 0
  )
  best_iter_pr <- max(cv_res$evaluation_log$test_aucpr_mean)
  if (best_iter_pr > best_xgb_pr) {
    best_xgb_pr <- best_iter_pr
    best_xgb_params <- xgb_grid[i, ]
  }
}
models$XGBoost <- xgb.train(
  params = list(
    objective = "binary:logistic", eval_metric = "aucpr",
    eta = best_xgb_params$eta, max_depth = best_xgb_params$max_depth
  ),
  data = dtrain_xgb, nrounds = 500, verbose = 0
)

# 3. LightGBM (Grid Search)
set.seed(1)
lgb_grid <- expand.grid(learning_rate = c(0.01, 0.05), num_leaves = c(15, 31))
best_lgb_pr <- 0
best_lgb_params <- list()
for (i in 1:nrow(lgb_grid)) {
  cv_res <- lgb.cv(
    params = list(
      objective = "binary", metric = "average_precision",

```

```

        learning_rate = lgb_grid$learning_rate[i],
        num_leaves = lgb_grid$num_leaves[i]
    ),
    data = lgb_train, nrounds = 500, nfold = 5,
    early_stopping_rounds = 50, verbose = -1
)
if (cv_res$best_score > best_lgb_pr) {
    best_lgb_pr <- cv_res$best_score
    best_lgb_params <- lgb_grid[i, ]
}
}
models$LightGBM <- lgb.train(
    params = list(
        objective = "binary", metric = "average_precision",
        learning_rate = best_lgb_params$learning_rate,
        num_leaves = best_lgb_params$num_leaves
    ),
    data = lgb_train, nrounds = 500, verbose = -1
)

# 4. Random Forest (ranger with tuneGrid)
set.seed(1)
models$`Random Forest` <- train(
    undocu_likely ~ .,
    data = caret_train, method = "ranger",
    metric = "AUC", trControl = caret_ctrl,
    tuneGrid = expand.grid(
        mtry = c(2, 4, 6), splitrule = "gini", min.node.size = c(1, 5)
    ),
    num.trees = 500, importance = "impurity"
)

# 5. Caret Models (GBM, GLM, KNN, AdaBoost) with Explicit Grids
set.seed(1)
models$GBM <- train(
    undocu_likely ~ .,
    data = caret_train,
    method = "gbm",
    metric = "AUC",
    trControl = caret_ctrl,
    verbose = FALSE,
    tuneGrid = expand.grid(

```

```

      n.trees = c(100, 300, 500),
      interaction.depth = c(3, 5, 7),
      shrinkage = c(0.01, 0.05),
      n.minobsinnode = c(10, 20)
    )
  )

models$`Logistic Regression` <- train(
  undocu_likely ~ .,
  data = caret_train, method = "glmnet",
  metric = "AUC", trControl = caret_ctrl,
  tuneGrid = expand.grid(
    alpha = seq(0, 1, 0.5), lambda = 10^seq(-3, -1, length = 5)
  )
)

models$KNN <- train(
  undocu_likely ~ .,
  data = caret_train, method = "knn",
  metric = "AUC", trControl = caret_ctrl,
  preProcess = c("center", "scale"),
  tuneGrid = expand.grid(k = c(5, 9, 15, 21))
)

models$AdaBoost <- train(
  undocu_likely ~ .,
  data = caret_train, method = "ada",
  metric = "AUC", trControl = caret_ctrl,
  tuneGrid = expand.grid(
    iter = c(50, 100, 150), maxdepth = c(2, 4, 6), nu = c(0.01, 0.1, 1)
  )
)

cat("Successfully trained and grid-searched all 8 models.\n")

```

Successfully trained and grid-searched all 8 models.\n

5. DIAGNOSTICS: COMPREHENSIVE METRICS ACROSS CONTEXTS

```

# =====
# 5. DIAGNOSTICS: COMPREHENSIVE METRICS ACROSS CONTEXTS
# =====

# 1. Shared Helper Functions for Predictions and Evaluation
generate_preds <- function(algo, model_obj, cat_pool, xgb_mat, lgb_mat, caret_df) {
  if (algo == "CatBoost") {
    return(catboost.predict(model_obj, cat_pool, prediction_type = "Probability"))
  }
  if (algo == "XGBoost") {
    return(predict(model_obj, newdata = xgb_mat))
  }
  if (algo == "LightGBM") {
    return(predict(model_obj, newdata = lgb_mat))
  }
  # Caret prediction method (Random Forest, GBM, GLM, KNN, AdaBoost)
  return(predict(model_obj, newdata = caret_df, type = "prob")[, "Class1"])
}

calc_brier <- function(preds, actuals) {
  mean((preds - actuals)^2, na.rm = TRUE)
}

calc_logloss <- function(preds, actuals) {
  eps <- 1e-15
  preds <- pmax(pmin(preds, 1 - eps), eps)
  -mean(actuals * log(preds) + (1 - actuals) * log(1 - preds), na.rm = TRUE)
}

calc_prauc <- function(preds, actuals) {
  perf_obj <- performance(prediction(preds, actuals), "prec", "rec")
  x <- perf_obj@x.values[[1]]
  y <- perf_obj@y.values[[1]]
  idx <- order(x)
  x <- x[idx]
  y <- y[idx]
  sum(diff(x) * (y[-1] + y[-length(y)]) / 2, na.rm = TRUE)
}

# 2. Format full Out-of-Sample Test set
x_test <- sipp_test %>% select(all_of(features))
y_test_num <- as.numeric(sipp_test$undocu_likely)

```

```

pool_test <- catboost.load_pool(data = x_test, label = y_test_num)
caret_test <- x_test %>%
  mutate(undocu_likely = factor(
    y_test_num,
    levels = c(1, 0), labels = c("Class1", "Class0")
  ))
mat_test <- predict(dummy_model, newdata = x_test)
dtest_xgb <- xgb.DMatrix(data = mat_test, label = y_test_num)

robustness_tracker <- data.frame()

# 3. Generate Diagnostics
for (algo in names(models)) {
  # Predict across contexts
  p_in <- generate_preds(
    algo, models[[algo]], pool_train, dtrain_xgb, mat_train, caret_train
  )
  p_out <- generate_preds(
    algo, models[[algo]], pool_test, dtest_xgb, mat_test, caret_test
  )
  p_drift <- generate_preds(
    algo, models[[algo]], pool_valid, dvalid_xgb, mat_valid, caret_valid
  )

  # Calculate Metrics
  robustness_tracker <- rbind(robustness_tracker, data.frame(
    Algorithm = algo,
    Drift_PRAUC = calc_prauc(p_drift, y_valid_num),
    Drift_ROCAUC = as.numeric(pROC::auc(pROC::roc(y_valid_num, p_drift, quiet = TRUE))),
    Drift_Brier = calc_brier(p_drift, y_valid_num),
    Drift_LogLoss = calc_logloss(p_drift, y_valid_num),
    Overfit_Penalty = calc_prauc(p_in, y_train_num) - calc_prauc(p_out, y_test_num),
    Shift_Penalty = calc_prauc(p_out, y_test_num) - calc_prauc(p_drift, y_valid_num)
  ))
}

robustness_tracker %>%
  arrange(desc(Drift_PRAUC)) %>%
  kable(digits = 4, caption = "Diagnostics (Sorted by Drift PR AUC)")

```

Table 1: Diagnostics (Sorted by Drift PR AUC)

Algorithm	Drift_PRAUC	Drift_ROCAUC	Drift_Brier	Drift_LogLoss	Overfit_Penalty	Shift_Penalty
Random Forest	0.5559	0.7374	0.1866	0.5484	0.4056	0.0159
CatBoost	0.5511	0.7235	0.1915	0.5670	0.3453	0.0164
XGBoost	0.5271	0.7106	0.1954	0.5775	0.2625	0.0092
GBM	0.5210	0.7220	0.1913	0.5621	0.1112	0.0144
AdaBoost	0.5153	0.7097	0.2043	0.5980	0.1275	0.0140
Logistic Regression	0.5130	0.7146	0.1935	0.5663	-0.0029	0.0134
KNN	0.5101	0.6991	0.1973	0.5981	0.0963	0.0118
LightGBM	0.5093	0.7045	0.1986	0.5896	0.3483	0.0190

6. VALIDATE TARGET-LIKE SAMPLE & GENERATE THRESHOLDS

```
# =====
# 6. VALIDATE TARGET-LIKE SAMPLE & GENERATE THRESHOLDS
# =====

val_results <- data.frame(
  Algorithm = character(),
  PR_AUC = numeric(),
  Threshold_75 = numeric()
)
thresholds <- list()

for (algo in names(models)) {
  # Predict on Target-Like validation set
  preds <- generate_preds(
    algo, models[[algo]], pool_valid, dvalid_xgb, mat_valid, caret_valid
  )

  pr_auc_score <- calc_prauc(preds, y_valid_num)

  # Calculate top 75% high-recall threshold
  valid_df <- tibble(p = preds, y = y_valid_num)
  t_75 <- min((valid_df %>%
    filter(y == 1) %>%
    arrange(desc(p)) %>%
    slice(1:floor(0.75 * n()))$p)
```

```

val_results <- rbind(val_results, data.frame(
  Algorithm = algo, PR_AUC = pr_auc_score, Threshold_75 = t_75
))
thresholds[[algo]] <- t_75
}

val_results <- val_results %>% arrange(desc(PR_AUC))
kable(val_results, digits = 4, caption = "Target-Like SIPP Validation Metrics")

```

Table 2: Target-Like SIPP Validation Metrics

Algorithm	PR_AUC	Threshold_75
Random Forest	0.5559	0.2989
CatBoost	0.5511	0.2458
XGBoost	0.5271	0.2545
GBM	0.5210	0.2758
AdaBoost	0.5153	0.3618
Logistic Regression	0.5130	0.3059
KNN	0.5101	0.2857
LightGBM	0.5093	0.2487

7. SCORE ACS TARGET SAMPLE AND EXPORT DATA

```

# =====
# 7. SCORE ACS TARGET SAMPLE (WINNING MODEL ONLY)
# =====
best_model_name <- val_results$Algorithm[1]
cat("Scoring ACS dataset using the winning model:", best_model_name, "\\n")

```

Scoring ACS dataset using the winning model: Random Forest \n

```

acs_export <- acs %>% select(year, serial, pernum)

# Generate continuous probabilities for ACS using ONLY the best model
acs_probs <- generate_preds(
  best_model_name, models[[best_model_name]], pool_acs, dacs_xgb, mat_acs, caret_acs
)

```

```
acs_export$winning_model_p <- acs_probs
acs_export$winning_model_high_recall <- ifelse(
  acs_probs >= thresholds[[best_model_name]], 1, 0
)
```

8. DIAGNOSTIC SANITY CHECKS (WINNING MODEL)

```
# =====
# 8. DIAGNOSTIC SANITY CHECKS (WINNING MODEL)
# =====
# Generate SIPP predictions for the winning model to compare alongside ACS
p_test <- generate_preds(
  best_model_name, models[[best_model_name]], pool_test, dtest_xgb, mat_test, caret_test
)
p_valid <- generate_preds(
  best_model_name, models[[best_model_name]], pool_valid, dvalid_xgb, mat_valid, caret_valid
)

# --- Sanity Check 1: Probability Distribution Percentiles ---
# Aggressively shortened labels to prevent LaTeX PDF overlap
dist_acs <- tibble(Sample = "ACS", p = acs_probs)
dist_test <- tibble(Sample = "SIPP (All)", p = p_test)
dist_targetlike <- tibble(Sample = "SIPP (Target)", p = p_valid)

dist_all <- bind_rows(dist_acs, dist_test, dist_targetlike) %>%
  group_by(Sample) %>%
  summarise(
    N = n(),
    Mean = mean(p),
    P10 = quantile(p, 0.10),
    P25 = quantile(p, 0.25),
    P50 = quantile(p, 0.50),
    P75 = quantile(p, 0.75),
    P90 = quantile(p, 0.90)
  ) %>%
  arrange(match(Sample, c("ACS", "SIPP (All)", "SIPP (Target)")))

kable(dist_all,
  digits = 4,
```

```
caption = paste("Sanity Check 1:", best_model_name, "Probability Distributions")
)
```

Table 3: Sanity Check 1: Random Forest Probability Distributions

Sample	N	Mean	P10	P25	P50	P75	P90
ACS	185845	0.2422	0.0741	0.1281	0.216	0.3220	0.4566
SIPP (All)	1392	0.3375	0.1111	0.1871	0.310	0.4856	0.5912
SIPP (Target)	1283	0.3320	0.1098	0.1819	0.303	0.4796	0.5850

```
# --- Sanity Check 2: Target-Like SIPP Lift by Quartile ---
base_rate_val <- mean(y_valid_num)

lift_table <- tibble(p = p_valid, truth = y_valid_num) %>%
  mutate(quartile = ntile(p, 4)) %>%
  group_by(quartile) %>%
  summarise(
    N = n(),
    Pos = sum(truth),
    Precision = mean(truth),
    Mean_P = mean(p)
  ) %>%
  mutate(
    Base_Rate = base_rate_val,
    Lift = Precision / Base_Rate,
    Q = paste0("Q", quartile)
  ) %>%
  select(Q, N, Pos, Precision, Mean_P, Base_Rate, Lift)

kable(lift_table,
  digits = 4,
  caption = paste("Sanity Check 2:", best_model_name, "Lift by Quartile")
)
```

Table 4: Sanity Check 2: Random Forest Lift by Quartile

Q	N	Pos	Precision	Mean_P	Base_Rate	Lift
Q1	321	31	0.0966	0.1176	0.325	0.2971
Q2	321	77	0.2399	0.2420	0.325	0.7380
Q3	321	128	0.3988	0.3886	0.325	1.2269

Q	N	Pos	Precision	Mean_P	Base_Rate	Lift
Q4	320	181	0.5656	0.5807	0.325	1.7403

```
# --- Sanity Check 3: Dynamic Feature Importance ---
# Extracts feature importance dynamically with fallback
get_var_imp <- function(algo, model_obj, feature_names) {
  tryCatch(
    {
      # Force load gbm so Caret can find the relative.influence function
      require(gbm)

      if (algo == "CatBoost") {
        imp <- catboost.get_feature_importance(model_obj)
        return(
          tibble(Feature = feature_names, Importance = as.numeric(imp)) %>%
            arrange(desc(Importance))
        )
      } else if (algo == "XGBoost") {
        imp <- xgb.importance(feature_names = feature_names, model = model_obj)
        return(as_tibble(imp) %>% select(Feature, Importance = Gain))
      } else if (algo == "LightGBM") {
        imp <- lgb.importance(model_obj, percentage = TRUE)
        return(as_tibble(imp) %>% select(Feature, Importance = Gain))
      } else {
        # Caret models (RF, GBM, GLM, KNN, AdaBoost)
        imp_obj <- varImp(model_obj, scale = FALSE)
        imp_df <- imp_obj$importance
        imp_df$Feature <- rownames(imp_df)
        if ("Overall" %in% colnames(imp_df)) {
          res <- as_tibble(imp_df) %>% select(Feature, Importance = Overall)
        } else {
          res <- as_tibble(imp_df) %>% select(Feature, Importance = 1)
        }
        return(res %>% arrange(desc(Importance)))
      }
    },
    error = function(e) {
      return(tibble(Feature = c("Extraction Failed", e$message), Importance = NA))
    }
  )
}
```

```

top_features <- get_var_imp(best_model_name, models[[best_model_name]], features)
kable(head(top_features, 10),
  digits = 4,
  caption = paste("Sanity Check 3: Top Features for", best_model_name)
)

```

Table 5: Sanity Check 3: Top Features for Random Forest

Feature	Importance
age	189.0312
years_us	176.6430
household_size	115.2534
yrsed	102.2530
central_latino	56.8219
cit_spouse	33.1565
poverty	31.8853
spanish_hispanic_latino	28.6922
fem	28.6568
nonfluent	28.3060

9. AUTOMATED SAVE AND ENVIRONMENT LOCKING

```

# =====
# 9. AUTOMATED SAVE AND ENVIRONMENT LOCKING
# =====

# 1. Export Data
write_csv(acs_export, acs_prediction_export)

# 2. Export ALL Models & Best Metadata
all_models_file <- file.path(ml_output_path, "all_optimized_models.rds")
best_metadata_file <- file.path(ml_output_path, "best_ml_metadata.rds")

final_metadata <- list(
  algorithm = best_model_name,
  report_timestamp = report_timestamp,
  features = features,
  pr_auc = val_results$PR_AUC[val_results$Algorithm == best_model_name],
  threshold_75 = thresholds[[best_model_name]]
)

```

```
)

# Saving the entire models list for later evaluation
saveRDS(models, all_models_file)
saveRDS(final_metadata, best_metadata_file)

# 3. Environment & Style Locking
renv::snapshot(prompt = FALSE)
```

- The lockfile is already up to date.

```
styler::style_file("ML_Models.qmd")
```

Styling 1 files:

ML_Models.qmd v

```
-----
Status Count Legend
v 1 File unchanged.
i 0 File changed.
x 0 Styling threw an error.
-----
```

```
tibble(
  Exported_Item = c("ACS Predictions", "All 8 Models Bundle", "Metadata"),
  Status = rep("Saved", 3)
) %>% kable()
```

Exported_Item	Status
ACS Predictions	Saved
All 8 Models Bundle	Saved
Metadata	Saved